```
}
int main()
{
        return UnitTest::RunAllTests();
}
```

In the code above, the *dataFixture* class generates a number that is one higher or lower than the passed number (this is the data generation part). The *SUITE(<suitename>)* macro embeds the set of test cases into a single suite. The *TEST_FIXTURE(<datafixture>, <testname>)* macro uses the data fixture class to obtain the data to be used in the test. The *TEST(<testname>)* macro is used for simple tests. *CHECK* or *CHECK_EQUAL* macros are used for comparing the results.

*UnitTest++* also provides macros for boundary checking, condition assertions, a timed constraint test and exception checking: *UNITTEST_TIME_CONSTRAINT, UNITTEST_TIME_CONSTRAINT_EXEMPT, CHECK_CLOSE, CHECK_THROW, CHECK_ARRAY_EQUAL, CHECK_ARRAY_CLOSE*, and so on. You can explore the *UnitTest++/docs* directory for more information.

Here is the *Makefile* that we will use to build the *test_ut.bin* binary, which is linked with *UnitTest++* and the *gcov* library. The various *make* targets provide for testing builds as well as release builds, prior to distributing the application to users. The compilation options *-fprofile-arcs* and *-ftest-coverage* are needed for code coverage checking, which we will discuss in the next section.

```
bash> cat $TESTROOT/Makefile
DEFAULT : all
CC=gcc
CXX=g++
CROSS_COMPILE=arm-linux-
TCC=${CROSS_COMPILE}${CC}
RM = rm

release.o :
        ${TCC} -c ${SRC_ROOT}/test.c ${SRC_ROOT}/main.c -I ${SRC_ROOT}

release_cov_valgrind.o :
        ${CC} -fprofile-arcs -ftest-coverage -c ${SRC_ROOT}/test.c ${SRC_
ROOT}/main.c -I ${SRC_ROOT}

test_cov.o :
        ${CC} -g -fprofile-arcs -ftest-coverage -c ${SRC_ROOT}/test.c -I
${SRC_ROOT}

test.o :
        ${CC} -g -c ${SRC_ROOT}/test.c -I ${SRC_ROOT}

unittest : test.o
${CXX} test.o ${TEST_ROOT}/testUt.cpp -o test_ut.bin -I ${SRC_ROOT}
-I ${TOOLS_ROOT}/UnitTest++/src/ -L${TOOLS_ROOT}/UnitTest++ -lUnitTest++

unittest_cov : test_cov.o
        ${CXX} test.o ${TEST_ROOT}/testUt.cpp -o test_ut.bin -I
${SRC_ROOT} -I ${TOOLS_ROOT}/UnitTest++/src/  -L${TOOLS_ROOT}/UnitTest++
-lUnitTest++ -lgcov
release : release.o
        ${TCC}  test.o main.o -o release.bin -I ${SRC_ROOT}

release_cov_valgrind : release_cov_valgrind.o
        ${CC}  test.o main.o -o release_cov_valgrind.bin -I ${SRC_ROOT}
-lgcov

all : unittest_cov

clean:
        -@${RM} *.o *.bin *.html *.gcda* *.gcno* *.info* *.png *.bin *.css
2>/dev/null
```

Let's compile the code and run the test:

```
bash> cd $TEST_ROOT
bash> make unittest
bash> ./test_ut.bin

Success: 4 tests passed.
Test time: 0.00 seconds.
```

You can play around with conditional assertions to get different results.

## Viewing code coverage

*lcov* is an extension of *gcov*, a GNU test coverage tool. lcov code coverage is used to examine the parts of the source code that are executed—the branches taken, and so on. It also gives us an execution count for each line of the source code.

To get the code coverage, in the *Makefile*, we added the *fprofile-arcs* option to instrument the program flow, and thus record how many times each function call, branch or line is executed. During the program run (execution of the test binary *test_ut.bin*), the generated information is saved in a *.gcda* file. The *ftest-coverage* option we added in the *Makefile* generates the test coverage note files *.gcno* files for coverage analysis.

The *geninfo* command converts the coverage data files into trace files, which are encoded ASCII text files containing information about the file location, functions, branch coverage, frequency of execution and so on. The *genhtml* command can then convert these to a readable HTML output (it creates a file named *index.html*):

```
bash> make unittest_cov
bash> geninfo .
bash> genhtml test.gcda.info
```